CS3500 Operating Systems
Copy-on-Write and Memory-Mapped Files Lab

Due: 11:59 PM, Sunday, 4 October 2026

1 Overview

In this lab you will extend xv6's virtual-memory system in two related ways. First, you will implement copy-on-write (COW) `fork()`, so that a parent and child initially share physical pages instead of copying them eagerly. Second, you will add a small, file-backed subset of the UNIX `mmap()` and `munmap()` interfaces. File pages must be loaded only when a process first accesses them.

Both parts use page faults. Your handler must distinguish COW faults, accesses to file-backed regions that have not been loaded, and invalid accesses. The two features must also work together across `fork()`.

2 Background Reading and Environment Setup

Before starting, read Chapters 3 and 4 of the xv6 book and review the following files:

- `kernel/vm.c`, `kernel/kalloc.c`, and `kernel/riscv.h`;
- `kernel/trap.c`, especially `usertrap()`;
- `kernel/proc.c` and `kernel/proc.h`; and
- `kernel/file.c`, `kernel/file.h`, and `kernel/fs.c`.

Use the lab repository announced on Moodle. The repository contains one common starter tree; this assignment is not built on your Page Tables Lab implementation. After cloning it, use the same xv6 Docker environment as in the previous labs:

```
$ git clone https://github.com/Lime787/CS3500-COW-Mmap-Lab
$ docker run -it -v <path-to-CS3500-COW-Mmap-Lab>:/home/xv6-labs \
    nandhagk/xv6-tools:latest
```

Run `make qemu` once before making changes. The unmodified starter should boot and should pass `usertests -q`. The new COW and mapping tests are expected to fail initially.

3 Copy-on-Write Fork

The original xv6 `fork()` copies every user page. This uses time and memory even when the child soon calls `exec()` and discards those pages.

A COW fork maps the same physical pages into the parent and child. Pages that were writable are made read-only and marked as COW in both page tables. A later write raises a page fault. The faulting process then receives a writable private copy while the other process retains the old page. Pages that were originally read-only remain ordinary read-only pages: attempting to write them is an error, not a COW operation.

3.1 Physical-page reference counts

A shared page must remain allocated until its last mapping is removed. Maintain a reference count for each physical page. Increase it when `fork()` adds a mapping and decrease it when a mapping is removed. Return the page to the free list only when the count reaches zero. A fixed-size array indexed by physical page address is sufficient; size it for the pages managed by `kinit()`.

3.2 COW page faults

Modify `uvmcopy()` and the page-fault path so that:

1. writable user pages are shared, marked COW, and made read-only in both processes;
2. originally read-only pages are shared without being marked COW;
3. a write fault on a COW page gives the faulting process a writable private page;
4. a write to a genuinely read-only page kills the process; and
5. allocation failure during fault handling kills the process cleanly.

You may use the RISC-V PTE software bits to mark COW mappings. The flag macros are near the end of `kernel/riscv.h`.

3.3 Kernel writes to user memory

The kernel can modify user memory without causing a user-mode store fault. Update `copyout()` so that a kernel write to a COW destination has the same result as a user write: the destination process receives a writable private copy. Do not grant write access to an ordinary read-only mapping.

4 File-Backed Memory Mapping

You will implement the following subset of the UNIX interfaces:

```
void *mmap(void *addr, size_t len, int prot, int flags,
           int fd, off_t offset);
int munmap(void *addr, size_t len);
```

For this lab, the following restrictions apply:

- `addr` and `offset` are always zero;
- `len` is positive and may differ from the file length;
- `prot` is `PROT_READ`, `PROT_WRITE`, or both;
- `flags` is exactly one of `MAP_PRIVATE` or `MAP_SHARED`;
- at most 16 VMAs need to exist in one process;
- `munmap()` may remove an entire VMA, a prefix, or a suffix, but will not create a hole in the middle of a VMA; and
- the tests will not call `exec()` while a process has an active VMA.

`mmap()` returns the address selected by the kernel, or `(void *)-1` on failure. A writable `MAP_SHARED` mapping requires a file opened for writing.

4.1 Virtual-memory areas

A *virtual-memory area* (VMA) is a kernel record for one contiguous mapped range. It describes the range even when its pages have not been loaded. The page-fault handler uses the VMA to find the backing file and the permissions for an address.

Keep a fixed-size array of VMAs in each process. One possible VMA record is:

```
struct vma {
  int used;
  uint64 addr;
  uint64 len;
  int prot;
  int flags;
  uint64 offset;
  struct file *file;
};
```

A size of 16 entries is sufficient. `mmap()` must choose an unused, page-aligned virtual address range. You may reserve the range starting at `0x40000000` for mappings. The mapping tests will not grow the heap into this range.

The VMA must retain its backing `struct file` even if the process closes the original file descriptor. Use the existing file-reference operations. Release that reference exactly once when the last part of the VMA disappears.

4.2 Lazy population

`mmap()` must create only the VMA; it must not allocate pages or read the file. On the first access to a page in the VMA, the fault handler must:

1. allocate and clear one physical page;
2. read the corresponding file range into that page;
3. zero any bytes not supplied by the file; and
4. map the page with permissions derived from the VMA.

Take the inode lock when required by the filesystem routine. Kill the process on an invalid access, a permission violation, or an allocation failure.

4.3 Unmapping and writeback

`munmap()` removes loaded pages in the requested range and updates the VMA. It must also work when some or all pages in the range have never been populated.

Write `MAP_SHARED` pages back to the file before unmapping them. Never write `MAP_PRIVATE` pages back. You may write every loaded shared page; checking the dirty bit is not required.

Process exit must release every mapping, page reference, and file reference. It must write back shared mappings as if they had been unmapped.

4.4 Hints

- The system-call declarations, numbers, dispatch entries, user stubs, constants, and kernel stubs are provided. Replace the stubs with your implementation.
- Review `filedup()`, `fileclose()`, `readi()`, and `filewrite()`.
- `uvmunmap()` can remove populated pages from a mapping.

5 Interaction Between COW and Mappings

On `fork()`, copy the parent's VMA records to the child and retain each backing file. Handle mapped pages as shown in Table 1.

Page at `fork()`	Required behavior
Loaded `MAP_PRIVATE` page	Preserve its current contents in the child and use COW for a writable page. Later writes by either process are private.
Loaded `MAP_SHARED` page	Share the physical page and retain the VMA permissions; writes are visible through both mappings.
Unloaded VMA page	Leave it unmapped and load it on first access.

Table 1: Required mapping behavior across `fork()`.

Shared-page limit

If a shared page was not loaded before `fork()`, the parent and child may later load separate physical pages for it. The tests will not require immediate visibility between those separate copies or perform conflicting writes through them. Writeback on `munmap()` and exit is still required.

6 Testing and Checkpoint

By the end of the first week, your implementation should pass `cowtest`. As you build `mmap()`, run `mmaptest` after adding VMA records, lazy page loading, unmapping, and fork support; each stage should get further through the test. Before submission, run:

```
$ make grade
```

This runs `cowtest`, `mmaptest`, `vmtest`, and `usertests -q`, and checks for `reference.txt`. Additional tests may check any required behavior in this handout.

7 Submission

7.1 References and collaboration

Create `reference.txt`. List the sources you consulted and any students with whom you collaborated, including the specific topics discussed. If you used generative AI to understand, debug, or generate any portion of your work, state where it was used and include the relevant prompt logs.

7.2 Submitting

Commit every file needed by your solution, then run:

```
$ make grade
$ make zipball
```

Untracked files are not included in the zip archive. Inspect `git status` and the messages printed by `make zipball`. Rename the archive to `<ROLL_NUMBER>_OS_LAB_4_COW_MMAP.zip` and upload it to Moodle. The submitted archive must build in the provided environment.

8 Acknowledgement

This assignment is adapted from the MIT 6.1810 Fall 2025 Copy-on-Write Fork and mmap labs. The xv6 teaching materials and lab framework were developed by the MIT PDOS group. The two components have been combined and their interaction specified for CS3500.